

API

Application Programming Interface

Guida Completa — Storia, Implementazione, Sicurezza e Futuro

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

SOMMARIO

[Clicca per navigare](#)

1. Origini e Storia delle API	5
1.1 Cosa è una API: Definizione e Concetto	5
1.2 Anni '60-'80: Le Prime API di Sistema.....	5
1.3 Anni '90: DCOM, CORBA e l'Era Enterprise	6
1.4 Fine anni '90 - 2000: SOAP e i Web Service XML	6
1.5 2000-2010: REST e la Rivoluzione del Web 2.0	7
1.6 2010-2020: GraphQL, gRPC, OpenAPI e la Maturità.....	7
1.7 2020-2026: Stato dell'Arte e Tendenze Attuali	8
2. Tipologie di API: REST, GraphQL, gRPC, SOAP e Oltre	10
2.1 REST API: Principi, Vantaggi e Limiti.....	10
2.2 GraphQL: Query Language per API.....	10
2.3 gRPC: Remote Procedure Calls ad Alta Performance	11
2.4 WebSocket, SSE e API Real-Time.....	12
2.5 Tabella Comparativa: REST vs GraphQL vs gRPC vs SOAP.....	12
3. Progettazione di una API: Principi e Best Practices	14
3.1 API Design First: Il Contratto Prima del Codice.....	14
3.2 Versionamento delle API	14
3.3 Rate Limiting, Throttling e Quota.....	15
4. Implementazione Python: Creare API con FastAPI	17
4.1 Setup e Primo Endpoint FastAPI.....	17
4.2 API CRUD Completa con Database.....	18
4.3 Autenticazione e Autorizzazione nella API	20
4.4 Rate Limiting, Middleware e Logging	22
5. Progetto Pratico: Sistema IoT per Trattamento Acque	24
5.1 Architettura del Progetto.....	24
5.2 Codice Completo: Water Treatment API	24
5.3 Client Python: Consumare la API	28
6. Cybersicurezza delle API: OWASP Top 10 e Attacchi.....	31
6.1 Il Panorama della Sicurezza API nel 2024	31
6.2 BOLA/IDOR: L'Attacco più Comune alle API	31

6.3 Injection Attacks: SQL, NoSQL, Command Injection	32
6.4 Mass Assignment, Excessive Data Exposure e SSRF.....	33
6.5 Come gli Hacker Attaccano le API: Tecniche e Tool.....	35
6.6 Mitigazioni Complete: Defense in Depth per API	35
7. Alternative, Concorrenti e Futuro delle API	38
7.1 Alternative a REST: Quando Non Usarlo	38
7.2 API Gateway: Kong, AWS API Gateway, Apigee.....	38
7.3 Il Futuro delle API: AI, WebTransport e Trends 2026-2030	39
7.4 Testing delle API: Strategie e Strumenti.....	40
7.5 Deployment e Scalabilità della API.....	41
Appendice — Risorse e Strumenti	44

API

Application Programming Interface

Guida Completa — Storia, Implementazione, Sicurezza e Futuro

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

1.2 Anni '60-'80: Le Prime API di Sistema

Le radici delle API risalgono agli albori dell'informatica moderna. Negli anni '60, i primi sistemi batch mainframe richiedevano interfacce standardizzate per la comunicazione tra sottosistemi. IBM con i suoi OS/360 (1964) definì alcune delle prime interfacce di chiamata di sistema documentate.

Negli anni '70, con la nascita di Unix (Bell Labs, 1969-1973), nasce lo standard POSIX (Portable Operating System Interface) — una delle API di sistema più influenti della storia, ancora oggi alla base di Linux, macOS e tutti i sistemi Unix-like. Le system call Unix (read, write, fork, exec, open) sono una API elegante e duratura.

Il concetto di RPC (Remote Procedure Call) nasce nei primi anni '80 con il paper fondamentale di Birrell e Nelson (1984): l'idea rivoluzionaria è che una funzione possa essere chiamata su un sistema remoto con la stessa sintassi di una chiamata locale. Sun Microsystems implementa RPC nel 1984 per

API

Application Programming Interface

Guida Completa — Storia, Implementazione, Sicurezza e Futuro

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

1.4 Fine anni '90 - 2000: SOAP e i Web Service XML

La nascita del web commerciale nella seconda metà degli anni '90 porta con sé una realizzazione fondamentale: HTTP attraversa qualsiasi firewall. Se tutti i sistemi già supportano HTTP, perché non usarlo anche per la comunicazione machine-to-machine?

XML-RPC nasce nel 1998, proposto da Dave Winer e Microsoft: usa HTTP come trasporto e XML come formato di serializzazione. Semplice rispetto a CORBA, ma limitato.

API

Application Programming Interface

Guida Completa — Storia, Implementazione, Sicurezza e Futuro

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

1.6 2010-2020: GraphQL, gRPC, OpenAPI e la Maturità

Application Programming Interface

Guida Completa — Storia, Implementazione, Sicurezza e Futuro

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

1.7 2020-2026: Stato dell'Arte e Tendenze Attuali

Il panorama API nel 2026 è caratterizzato da convergenza, maturità e nuove sfide.

REST rimane dominante: Secondo il Postman State of API Report 2023, REST è usato dall'86% degli sviluppatori. Ma non è l'unica opzione e la scelta dipende sempre di più dal caso d'uso specifico.

API-First Design: Le organizzazioni moderne progettano le API prima del codice — il contratto API diventa il punto di partenza dello sviluppo. Strumenti come Stoplight, Postman, Insomnia permettono di mockare e testare API prima ancora di scrivere una riga di backend.

AI e API: L'esplosione dell'AI generativa (2023-2026) ha moltiplicato le API AI: OpenAI, Anthropic, Google Gemini, Mistral — tutte esposte via API REST. Gli LLM (Large Language Models) possono ora 'usare' API tramite function calling, creando agenti autonomi che orchestrano servizi via API.

WebTransport e HTTP/3: L'adozione di QUIC/HTTP/3 porta nuove capacità per API real-time con latenza ultrabassa.

Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

API

Application Programming Interface

Guida Completa — Storia, Implementazione, Sicurezza e Futuro

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

2.2 GraphQL: Query Language per API

GraphQL è un linguaggio di query e un runtime per API, sviluppato internamente da Facebook nel 2012 e rilasciato pubblicamente nel 2015. Non è un sostituto di REST ma un'alternativa con caratteristiche distinte.

CONCETTO FONDAMENTALE: In GraphQL, invece di endpoint multipli (/users, /posts, /comments), esiste un singolo endpoint (/graphql) attraverso cui il client invia una query che descrive esattamente i dati necessari.

Application Programming Interface

Guida Completa — Storia, Implementazione, Sicurezza e Futuro

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

2.3 gRPC: Remote Procedure Calls ad Alta Performance

gRPC (Google Remote Procedure Call) è un framework RPC open source sviluppato da Google, pubblicato nel 2016. Usa HTTP/2 come trasporto e Protocol Buffers (Protobuf) come linguaggio di serializzazione dei dati.

PROTOCOL BUFFERS: A differenza di JSON (testo), Protobuf usa serializzazione binaria. Il risultato è: 3-10x più veloce di JSON nella serializzazione. 40-60% più compatto di JSON. Schema obbligatorio: il .proto file definisce la struttura dei messaggi.

Esempio di definizione .proto: `syntax = 'proto3'; service UserService { rpc GetUser (UserRequest) returns (UserResponse); rpc ListUsers (ListRequest) returns (stream UserResponse); } message UserRequest { string user_id = 1; } message UserResponse { string id = 1; string name = 2; string email = 3; }`

MODALITA' gRPC: Unary: richiesta singola, risposta singola (come REST). Server Streaming: una richiesta, stream di risposte. Client Streaming: stream di richieste, risposta singola. Bidirectional Streaming: stream in entrambe le direzioni.

API

Application Programming Interface

Guida Completa — Storia, Implementazione, Sicurezza e Futuro

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

2.5 Tabella Comparativa: REST vs GraphQL vs gRPC vs SOAP

Confronto diretto tra le principali tecnologie API:

Guida Completa — Storia, Implementazione, Sicurezza e Futuro

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

API

Application Programming Interface

Guida Completa — Storia, Implementazione, Sicurezza e Futuro

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

3.2 Versionamento delle API

Il versionamento è uno dei problemi più dibattuti nel design API. Come evolvere una API senza rompere i client esistenti?

APPROCCI AL VERSIONAMENTO:

Guida Completa — Storia, Implementazione, Sicurezza e Futuro

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

3.3 Rate Limiting, Throttling e Quota

Il rate limiting è un meccanismo fondamentale per proteggere le API da abusi, garantire disponibilità equa tra i client e prevenire attacchi DoS.

ALGORITMI DI RATE LIMITING:

Fixed Window: Conta le richieste in una finestra temporale fissa (es: 1000 req/ora). Problema: burst al cambio della finestra (1000 req alle 23:59:59 + 1000 req alle 00:00:01 = 2000 req in 2 secondi).

Sliding Window: Conta le richieste nell'ultimo periodo mobile. Più preciso, più costoso in termini di memoria (richiede log dei timestamp).

Token Bucket: Un 'secchio' si riempie di token a velocità costante (es: 10 token/secondo, max 100 token). Ogni richiesta consuma un token. Permette burst controllati. Usato da AWS, Stripe.

Leaky Bucket: Le richieste entrano in una coda a velocità variabile, escono a velocità costante. Smooths out bursts completamente. Usato per QoS di rete.

HEADER STANDARD (RFC 6585, IETF Draft): X-RateLimit-Limit: 1000 X-RateLimit-Remaining: 743
X-RateLimit-Reset: 1735689600 Retry-After: 3600 (quando si supera il limite)

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

API

Application Programming Interface

Guida Completa — Storia, Implementazione, Sicurezza e Futuro

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

4.2 API CRUD Completa con Database

Esempio completo: API per gestione di una lista prodotti con SQLite.

[illegible]

[illegible]

API

Application Programming Interface

Guida Completa — Storia, Implementazione, Sicurezza e Futuro

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

4.3 Autenticazione e Autorizzazione nella API

La sicurezza è parte integrante del design API, non un'aggiunta successiva.

```
# auth_api.py
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import HTTPBearer, APIKeyHeader
from pydantic import BaseModel
import jwt, secrets, hashlib
from datetime import datetime, timedelta, timezone
from functools import wraps
```

[illegible]

```

async def get_jwt_user(
    credentials=Depends(bearer)
) -> dict:
    try:
        payload = jwt.decode(
            credentials.credentials,
            SECRET_KEY, algorithms=['HS256'])
    )
    return payload
except jwt.ExpiredSignatureError:
    raise HTTPException(401, 'Token scaduto')
except jwt.InvalidTokenError:
    raise HTTPException(401, 'Token non valido')

def require_scope(scope: str):
def dependency(user=Depends(get_jwt_user)):
    if scope not in user.get('scopes', []):

```

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

4.4 Rate Limiting, Middleware e Logging

Middleware essenziali per una API production-ready:

```
# middleware_api.py
from fastapi import FastAPI, Request, Response
from fastapi.middleware.cors import CORSMiddleware
from fastapi.middleware.gzip import GZipMiddleware
from starlette.middleware.base import BaseHTTPMiddleware
import time, logging, uuid
from collections import defaultdict
from datetime import datetime

app = FastAPI()

# ■■■ CORS ■■■
app.add_middleware(
    CORSMiddleware,
    allow_origins=['https://myapp.com', 'https://admin.myapp.com'],
    allow_credentials=True,
    allow_methods=['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],
    allow_headers=['Authorization', 'Content-Type', 'X-API-Key'],
)

# ■■■ COMPRESSION ■■■
app.add_middleware(GZipMiddleware, minimum_size=1000)

# ■■■ RATE LIMITER (Token Bucket in-memory) ■■■
class RateLimitMiddleware(BaseHTTPMiddleware):
    def __init__(self, app, max_requests=100, window=60):
        super().__init__(app)
        self.max_requests = max_requests
        self.window = window
        self.clients = defaultdict(list)

    async def dispatch(self, request: Request, call_next):
        client_ip = request.client.host
        now = time.time()
```

API

Application Programming Interface

Guida Completa — Storia, Implementazione, Sicurezza e Futuro

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

API

Application Programming Interface

Guida Completa — Storia, Implementazione, Sicurezza e Futuro

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

5.2 Codice Completo: Water Treatment API

```
# water_api/main.py
from fastapi import FastAPI, HTTPException, Depends, BackgroundTasks
from fastapi import Query, status
from pydantic import BaseModel, Field, validator
from typing import Optional, List
from datetime import datetime, timedelta
import sqlite3, uuid, asyncio

app = FastAPI()
```


[illegible]

[illegible]

```

)
db.commit()
if alerts:
    background_tasks.add_task(
        send_webhook_alert, reading.sensor_id, alerts, status
    )
return SensorResponse(
    **reading.dict(), id=reading_id,
    alerts=alerts, status=status
)

@app.get('/readings', response_model=List[dict])
async def get_readings(
    station_id: Optional[str] = None,

hours: int = Query(24, ge=1, le=720),
db: sqlite3.Connection = Depends(get_db)
):
    since = datetime.utcnow() - timedelta(hours=hours)
    query = 'SELECT * FROM readings WHERE timestamp > ?'
    params = [since]
    if station_id:
        query += ' AND station_id = ?'
        params.append(station_id)
    query += ' ORDER BY timestamp DESC'
    return [dict(r) for r in db.execute(query, params).fetchall()]

@app.get('/stats/{station_id}')
async def get_station_stats(
    station_id: str,
    db: sqlite3.Connection = Depends(get_db)
):
    row = db.execute('''
SELECT
AVG(ph) as avg_ph, MIN(ph) as min_ph, MAX(ph) as max_ph,
AVG(temperature) as avg_temp,
AVG(chlorine) as avg_chlorine,
AVG(flow) as avg_flow,
COUNT(*) as total_readings,
SUM(CASE WHEN status != 'normal' THEN 1 ELSE 0 END) as alerts
FROM readings WHERE station_id = ?
''', [station_id]).fetchone()
    return dict(row)

@app.get('/alerts', response_model=List[dict])
async def get_alerts(
    resolved: Optional[bool] = False,
    db: sqlite3.Connection = Depends(get_db)
):
    rows = db.execute(
        'SELECT * FROM alerts WHERE resolved = ? ORDER BY created_at DESC',
        [resolved]
    ).fetchall()
    return [dict(r) for r in rows]

async def send_webhook_alert(sensor_id, alerts, severity):
    import httpx

```

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

5.3 Client Python: Consumare la API

Un client completo che simula sensori e consuma la API:

```
# sensor_client.py
import httpx, asyncio, random, time
from datetime import datetime

API_BASE = 'http://localhost:8000'
API_KEY = 'key_abc123'

class WaterSensorSimulator:
    def __init__(self, sensor_id: str, station_id: str):
        self.sensor_id = sensor_id
        self.station_id = station_id
        # Valori base normali
        self.base_ph = 7.2
        self.base_temp = 18.0
        self.base_chlorine = 0.5
        self.base_flow = 150.0

    def generate_reading(self, anomaly: bool = False) -> dict:
        noise = lambda base, pct: base + random.gauss(0, base * pct)
        reading = {
            'sensor_id': self.sensor_id,
            'station_id': self.station_id,
            'ph': noise(self.base_ph, 0.02),
            'temperature': noise(self.base_temp, 0.03),
            'chlorine': noise(self.base_chlorine, 0.05),
            'flow': noise(self.base_flow, 0.04),
            'turbidity': abs(random.gauss(0.3, 0.1)),
            'timestamp': datetime.utcnow().isoformat(),
        }
        if anomaly:
```

```

param = random.choice(['ph', 'chlorine', 'turbidity'])
if param == 'ph':
    reading['ph'] = random.choice([5.8, 9.2])
elif param == 'chlorine':
    reading['chlorine'] = 0.05 # Troppo basso
elif param == 'turbidity':
    reading['turbidity'] = 3.5 # Troppo alta
return reading

async def run_sensor_loop(
    sensor: WaterSensorSimulator,
    interval: int = 5,
    anomaly_prob: float = 0.1
):
    async with httpx.AsyncClient(

base_url=API_BASE,
headers={'X-API-Key': API_KEY},
timeout=10.0
) as client:
    while True:
        is_anomaly = random.random() < anomaly_prob
        reading = sensor.generate_reading(anomaly=is_anomaly)
        try:
            resp = await client.post('/readings', json=reading)
            if resp.status_code == 201:
                data = resp.json()
                status = data.get('status', 'normal')
                emoji = '■' if status == 'critical' else (
                    '■■' if status == 'warning' else '■'
                )
                print(f'{emoji} [{sensor.sensor_id}] '
                    f'pH={reading["ph"]:.2f} '
                    f'Cl={reading["chlorine"]:.3f} '
                    f'Status: {status}')
                if data.get('alerts'):
                    for alert in data['alerts']:
                        print(f' ■ {alert}')
                else:
                    print(f'Errore: {resp.status_code} {resp.text}')
            except httpx.ConnectError:
                print('API non raggiungibile, riprovo...')
            await asyncio.sleep(interval)

        async def fetch_dashboard():
            async with httpx.AsyncClient(base_url=API_BASE) as client:
                stats = await client.get('/stats/STATION-A')
                alerts = await client.get('/alerts')
                print('=== DASHBOARD STAZIONE A ===')
                s = stats.json()
                print(f'pH medio: {s["avg_ph"]:.2f}')
                print(f'Temp media: {s["avg_temp"]:.1f}C')
                print(f'Lecture totali: {s["total_readings"]}')
                print(f>Allarmi attivi: {len(alerts.json())}')

        async def main():
            sensors = [

```



API

Application Programming Interface

Guida Completa — Storia, Implementazione, Sicurezza e Futuro

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

6.2 BOLA/IDOR: L'Attacco più Comune alle API

Broken Object Level Authorization (BOLA), anche noto come IDOR (Insecure Direct Object Reference), è il rischio #1 OWASP per le API. È semplice da sfruttare e devastante.

COME FUNZIONA L'ATTACCO: Un utente autenticato modifica un ID nei parametri della richiesta per accedere a dati di altri utenti. Il server verifica l'autenticazione (il token JWT è valido) ma NON verifica l'autorizzazione (l'utente ha diritto a QUEL specifico oggetto?).

Esempio vulnerabile:

API

Application Programming Interface

Guida Completa — Storia, Implementazione, Sicurezza e Futuro

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

6.3 Injection Attacks: SQL, NoSQL, Command Injection

Gli attacchi injection nelle API avvengono quando dati non validati dall'utente vengono passati direttamente a interpreti (SQL, shell, LDAP).

SQL INJECTION via API:

```
# VULNERABILE
@app.get('/search')
async def search_users(name: str):
```


API

Application Programming Interface

Guida Completa — Storia, Implementazione, Sicurezza e Futuro

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

6.4 Mass Assignment, Excessive Data Exposure e SSRF

MASS ASSIGNMENT (API3:2023 Broken Object Property Level Authorization): Avviene quando una API permette al client di aggiornare proprietà dell'oggetto che non dovrebbe poter modificare.

```
# VULNERABILE - Accetta qualsiasi campo
@app.put('/users/me')
async def update_profile(
    data: dict, # PERICOLO: dict non validato
    current_user=Depends(get_current_user)
):
    for key, value in data.items():
        setattr(current_user, key, value) # Pericolo!
    db.save(current_user)
# Attacco: {"is_admin": true, "role": "superuser"}

# SICURO - Schema esplicito con Pydantic
class UpdateProfileSchema(BaseModel):
    display_name: Optional[str] = None
    bio: Optional[str] = None
    # is_admin NON e' qui = non puo' essere modificato

@app.put('/users/me')
async def update_profile_safe(
    data: UpdateProfileSchema,
    current_user=Depends(get_current_user)
):
    update_data = data.dict(exclude_none=True)
    db.execute('UPDATE users SET ... WHERE id = ?',
        [*update_data.values(), current_user.id])
```

SSRF (Server Side Request Forgery, API7:2023): L'API fa una richiesta HTTP per conto del client verso un URL specificato. Un attaccante usa questo per accedere a risorse interne.

```
# VULNERABILE - Fetch URL arbitrario
@app.get('/fetch-url')
async def fetch_external(url: str):
    async with httpx.AsyncClient() as client:
        response = await client.get(url) # PERICOLO!
    return response.text
# Attacco: ?url=http://169.254.169.254/metadata (AWS metadata)
# Attacco: ?url=http://localhost:6379 (Redis interno)
# Attacco: ?url=file:///etc/passwd

# SICURO - Allowlist di domini
ALLOWED_DOMAINS = {'api.example.com', 'cdn.example.com'}
from urllib.parse import urlparse
@app.get('/fetch-url')
async def fetch_safe(url: str):
    parsed = urlparse(url)
    if parsed.hostname not in ALLOWED_DOMAINS:
        raise HTTPException(400, 'Dominio non permesso')
    if parsed.scheme not in ('http', 'https'):
        raise HTTPException(400, 'Schema non permesso')
    async with httpx.AsyncClient() as client:
        response = await client.get(url, timeout=5)
    return response.text
```

API

Application Programming Interface

Guida Completa — Storia, Implementazione, Sicurezza e Futuro

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

6.6 Mitigazioni Complete: Defense in Depth per API

La sicurezza delle API richiede un approccio stratificato. Nessuna singola misura è sufficiente.

```
# security_config.py – Configurazione sicurezza completa
from fastapi import FastAPI, Request, HTTPException
```

```
from fastapi.middleware.cors import CORSMiddleware
from fastapi.middleware.trustedhost import TrustedHostMiddleware
import re, hashlib, time

app = FastAPI()

# 1. HOST VALIDATION
app.add_middleware(
    TrustedHostMiddleware,
    allowed_hosts=['api.myapp.com', 'localhost']
)

# 2. CORS RESTRITTIVO
app.add_middleware(
    CORSMiddleware,
    allow_origins=['https://app.myapp.com'],
    allow_methods=['GET', 'POST', 'PUT', 'DELETE'],
    allow_headers=['Authorization', 'Content-Type'],
    allow_credentials=True,
    max_age=600,
)

# 3. SECURITY HEADERS
@app.middleware('http')
async def add_security_headers(request: Request, call_next):
    response = await call_next(request)
    response.headers['X-Content-Type-Options'] = 'nosniff'
    response.headers['X-Frame-Options'] = 'DENY'
    response.headers['X-XSS-Protection'] = '1; mode=block'
    response.headers['Strict-Transport-Security'] = (
        'max-age=31536000; includeSubDomains'
    )
    response.headers['Content-Security-Policy'] = (
        'default-src \'none\''
    )
    # Rimuovi header che rivelano la tecnologia
    del response.headers['server']
    return response

# 4. INPUT SANITIZATION
def sanitize_string(value: str, max_len: int = 500) -> str:
    value = value[:max_len]
    dangerous = ['<', '>', '"', "'", ';', '--', '/*', '*/']

    for d in dangerous:
        value = value.replace(d, '')
    return value.strip()

# 5. AUDIT LOGGING
import logging
audit_log = logging.getLogger('audit')

@app.middleware('http')
async def audit_middleware(request: Request, call_next):
    response = await call_next(request)
    if response.status_code >= 400:
        audit_log.warning(
```

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

API

Application Programming Interface

Guida Completa — Storia, Implementazione, Sicurezza e Futuro

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

7.2 API Gateway: Kong, AWS API Gateway, Apigee

Un API Gateway è un componente infrastrutturale che si interpone tra i client e i servizi backend, centralizzando funzionalità trasversali.

FUNZIONALITA' TIPICHE DI UN API GATEWAY: Routing: instrada le richieste ai servizi corretti. Autenticazione/Autorizzazione: verifica JWT, API Key centralmente. Rate Limiting: applica limiti a livello gateway prima di raggiungere i servizi. SSL Termination: gestisce TLS, i servizi interni ricevono HTTP. Load Balancing: distribuisce il traffico tra istanze. Caching: cache delle risposte a livello gateway. Logging/Monitoring: centralizza osservabilità. Transformation: modifica request/response (header injection, body transform).

Guida Completa — Storia, Implementazione, Sicurezza e Futuro

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

7.3 Il Futuro delle API: AI, WebTransport e Trends 2026-2030

Il panorama delle API è in rapida evoluzione. Queste sono le tendenze più significative per il prossimo quinquennio:

1. **AI-NATIVE API:** L'integrazione AI-API è già realtà (OpenAI, Anthropic, Google Gemini) ma diventerà ubiqua. Ogni API avrà uno strato AI che permette: Natural Language querying: 'Dammi le vendite dell'ultimo trimestre' invece di costruire query REST manualmente. Self-healing API: il sistema rileva errori di integrazione e si corregge. Semantic versioning: gli agenti AI capiscono le breaking change e adattano.
2. **MCP (Model Context Protocol, Anthropic 2024):** Protocollo standard per connettere LLM a strumenti e API. Permette agli AI assistant di usare API come tool in modo standardizzato. Potenzialmente il 'USB delle API per AI'.
3. **HTTP/3 e QUIC:** HTTP/3 su QUIC (UDP) riduce la latenza di connessione eliminando il TCP handshake. Adottato da Google, Cloudflare, Meta. Sarà lo standard per API ad alta performance entro 2027.
4. **CONFIDENTIAL COMPUTING API:** API che girano in Trusted Execution Environments (TEE) garantiscono che neanche il provider possa vedere i dati. Fondamentale per healthcare, finance, legal AI.

Python * FastAPI * AWS * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

7.4 Testing delle API: Strategie e Strumenti

Il testing delle API è fondamentale per garantire correttezza, performance e sicurezza. Ecco una suite di test completa in Python:

[illegible]

API

Application Programming Interface

Guida Completa — Storia, Implementazione, Sicurezza e Futuro

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

7.5 Deployment e Scalabilità della API

Una API va non solo scritta ma deployata e scalata correttamente.

```
# Dockerfile per FastAPI  
# FROM python:3.12-slim
```

```
# WORKDIR /app
# COPY requirements.txt .
# RUN pip install --no-cache-dir -r requirements.txt
# COPY . .
# EXPOSE 8000
# CMD ['uvicorn', 'main:app',
# '--host', '0.0.0.0',
# '--port', '8000',
# '--workers', '4']

# docker-compose.yml per sviluppo locale
# version: '3.9'
# services:
# api:
# build: .
# ports: ['8000:8000']
# environment:
# - DATABASE_URL=postgresql://...
# - SECRET_KEY=${SECRET_KEY}
# depends_on: [db, redis]
# db:
# image: postgres:16
# volumes: [postgres_data:/var/lib/postgresql/data]
# redis:
# image: redis:7-alpine
```

CONFIGURAZIONE UVICORN PRODUZIONE:

```
# gunicorn con workers uvicorn per produzione
# gunicorn main:app \
# --workers 4 \
# --worker-class uvicorn.workers.UvicornWorker \
# --bind 0.0.0.0:8000 \
# --timeout 60 \
# --access-logfile - \
# --error-logfile -
```

METRICHE E MONITORING:

```
# prometheus_client per FastAPI
from prometheus_client import Counter, Histogram, start_http_server
import time

REQUEST_COUNT = Counter(
    'api_requests_total',
    'Total API requests',
    ['method', 'endpoint', 'status']
)

REQUEST_LATENCY = Histogram(
    'api_request_duration_seconds',
    'API request latency',
    ['method', 'endpoint']
)

@app.middleware('http')
async def metrics_middleware(request, call_next):
    start = time.time()
    response = await call_next(request)
```

REST * GraphQL * gRPC * WebSocket * SOAP
Python * FastAPI * OWASP * Cybersecurity * IoT
API Gateway * Microservizi * Testing * Deployment

Versione 1.0 * 2026 * Solo Python * Con Esempi Pratici e Casi Reali

A

Appendice — Risorse e Strumenti

Appendice — Risorse e Strumenti

Risorsa	URL / Riferimento
RFC 7231 — HTTP/1.1 Semantics	https://datatracker.ietf.org/doc/html/rfc7231
OpenAPI Specification 3.1	https://spec.openapis.org/oas/v3.1.0
FastAPI Documentation	https://fastapi.tiangolo.com/
GraphQL Specification	https://graphql.github.io/graphql-spec/
gRPC Documentation	https://grpc.io/docs/
OWASP API Security Top 10	https://owasp.org/API-Security/
Postman State of API Report	https://www.postman.com/state-of-api/
Swagger UI	https://swagger.io/tools/swagger-ui/
jwt.io	https://jwt.io
Kong API Gateway	https://konghq.com/
httpx Python Client	https://www.python-httpx.org/
Pydantic Documentation	https://docs.pydantic.dev/
RFC 6455 — WebSocket	https://datatracker.ietf.org/doc/html/rfc6455
RFC 9000 — QUIC	https://datatracker.ietf.org/doc/html/rfc9000
Model Context Protocol	https://modelcontextprotocol.io/